```
    }
}
public class RectangleDemo {
    public static void main(String[] args) {
        Rectangle r = new Rectangle();
        r.length = 10;
        r.width = 5;
        r.display();
    }
}
```

Output:

```
Area: 50.0
Perimeter: 30.0
```

Experiment 5: Constructors

Program 5.1: Student Constructors

```
class Student {
    String name;
    int rollNo;
    Student() {
        name = "Unknown";
        rollNo = 0;
    }
    Student(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Roll Number: " + rollNo);
    }
```

```
}
public class StudentConstructor {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student("Raina", 101);
        System.out.println("Default Constructor:");
        s1.display();
        System.out.println("\nParameterized Constructor:");
        s2.display();
    }
}
```

Output:

```
Default Constructor:
Name: Unknown
Roll Number: 0
Parameterized Constructor:
Name: Raina
Roll Number: 101
```

Program 5.2: Constructor Overloading

```
class BankAccount {
    int accountNo;
    String name;
    double balance;
    BankAccount() {
        accountNo = 0;
        name = "Unknown";
        balance = 0;
    }
    BankAccount(int accountNo, String name) {
        this.accountNo = accountNo;
```

```
        this.name = name;
        balance = 0;
    }
    BankAccount(int accountNo, String name, double balance) {
        this.accountNo = accountNo;
        this.name = name;
        this.balance = balance;
    }
    void display() {
        System.out.println("Account Number: " + accountNo);
        System.out.println("Name: " + name);
        System.out.println("Balance: " + balance);
        System.out.println();
    }
}
public class BankAccountDemo {
    public static void main(String[] args) {
        BankAccount b1 = new BankAccount();
        BankAccount b2 = new BankAccount(101, "Raina");
        BankAccount b3 = new BankAccount(102, "Rinku", 25000);
        b1.display();
        b2.display();
        b3.display();
    }
}
```

Output:

```
Account Number: 0
Name: Unknown
Balance: 0.0
Account Number: 101
```

Name: Raina

Balance: 0.0

Account Number: 102

Name: Rinku

Balance: 25000.0

Experiment 6: The this Keyword

Program 6.1: Using this Keyword

```
class Student {
    String name;
    int age;
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
    void display() {
        System.out.println("Name: " + this.name);
        System.out.println("Age: " + this.age);
    }
}
public class ThisDemo {
    public static void main(String[] args) {
        Student s = new Student("Raina", 19);
        s.display();
    }
}
```

Output:

Name: Raina

Age: 19

Program 6.2: Constructor Chaining

```
class Student {
    String name;
    int age;
    String branch;
    Student() {
        this("Unknown");
    }
    Student(String name) {
        this(name, 0);
    }
    Student(String name, int age) {
        this(name, age, "CSE");
    }
    Student(String name, int age, String branch) {
        this.name = name;
        this.age = age;
        this.branch = branch;
    }
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Branch: " + branch);
    }
}
public class ConstructorChaining {
    public static void main(String[] args) {
        Student s = new Student("Raina", 19, "CSE-AIML");
        s.display();
    }
```

```
}
```

Output:

Name: Raina

Age: 19

Branch: CSE-AIML

Experiment 7: Garbage Collection

Program 7.1: Demonstrating Garbage Collection

```
class Demo {
    int id;
    Demo(int id) {
        this.id = id;
    }
    protected void finalize() {
        System.out.println("Object " + id + " is garbage collected.");
    }
}
public class GarbageCollectionDemo {
    public static void main(String[] args) {
        Demo d1 = new Demo(1);
        Demo d2 = new Demo(2);
        Demo d3 = new Demo(3);
        d1 = null;
        d2 = null;
        d3 = null;
        System.gc();
        System.out.println("Garbage collection requested.");
    }
}
```

Output:

Garbage collection requested.

Program 7.2: Object Eligibility for Garbage Collection

```
class Demo {
    int id;
    Demo(int id) {
        this.id = id;
    }
}
public class ObjectEligibility {
    public static void main(String[] args) {
        Demo d1 = new Demo(1);
        d1 = null;
        Demo d2 = new Demo(2);
        d2 = new Demo(3);
        new Demo(4);
        System.gc();
        System.out.println("Objects are eligible for garbage collection.");
    }
}
```

Output:

Objects are eligible for garbage collection.

Experiment 8: Method Overloading

Program 8.1: Overloading Arithmetic Methods

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
```